WallyTutor

What about Scientific Computing?

Walter Dal'Maz Silva

2026-08-17

Scientific Computing - not to be confused with High-Performance Computing (HPC) - is a branch of Computer Science highly intertwined with Physical Sciences and Engineering, and more recently Life Sciences. To put it simple, it concerns any use of a processor to perform a scientific task, whether that is simply plotting a cloud of data, analysing a time-series, sequencing DNA, or simulating a large-scale combustion turbine. In this note we will touch some of the topics related to Scientific Computing, specially in fields relevant the author's experience and teaching interests, mostly applied mathematics (thus certainly HPC), machine learning, and scientific software engineering. It will try to serve as a guide for introducing the field, and more specifically how to be efficient at producing reproducible Science, thus the *meta* aspects of the field.

The skill set

As of 2026, to be a well-rounded scientific computing practitioner, you need to be proficient in several technologies, including language models. Nonetheless, there are more classical skills that sometimes are overseen, but I deem mandatory.

> **i** We are limited beings
>
> It is humanly impossible to master everything at once; even after more than 10 years in the field as of today I only have a grasp in the tools I do not use everyday. Software and methods evolve, and unless you keep using a specific tool you simply cannot afford to keep up to date with it. Sometimes when looking at job offers I get overwhelmed by what recruiters are asking candidates purely out of arrogance. Wanting everything at once is the same as wanting nothing. But that should not be a roadblock for a scientist in the long term. As you get used to scientific software, getting back to a good level of some tool you used in the past is quick (it is true that it is not *extremely* fast in some cases) and learning new tools for which you already know the science behind is trivial. Even exploring new fields become easy in some cases.

The minimal skill set any scientist should have starts with version control, everything else is worthless without it, currently that means Git (other tools like SVN, Mercurial, and Darcs are

niche). You should be able to track the changes in your project over time, branch for testing new ideas, merge them back without complications, release versions, and so on. It should not be limited to software, but also notebooks, reports, everything should be thought to be produced as raw text so that version control works well. It is also the basis for collaboration and healthy interaction between team members. A subtle point I intentionally skipped here is the versioning of data. That is still a touchy subject for which classical version control tools - including Git - were not conceived to deal with, and no perfect solution exists nowadays.

Closely related to version control comes the topic of software documentation. Whether your production is intended to be used by yourself or others, the truth is that undocumented software cannot be taken seriously. Documenting code is not simply generating nicely formatted pages with tools such as Doxygen, Sphinx, Documenter.jl, etc., depending on how your code-base is written, but keeping meaningful comments at those points where the code is difficult to understand, explain *yourself to your future self* helping you remember how your code works when you come back to it later. It is essential for reproducibility and collaboration.

If you are versioning and documenting something, that something needs to be written somehow. Nowadays, in the scientific world that often means Python scripting, which is a mandatory item in any CV. But that is not enough! A low(er) level programming language among Rust, C, C++, and Fortran, preferably all of them, is also required. As you might have noticed, I said *Python scripting*, not *Python programming*, and that is related to why someone also needs these lower level programming skills. Plain Python is mostly useless for the scientist, it is way too slow. The *Python* used in the scientific world is just the surface of a large ecosystem writen mostly in C, some interfaces with legacy Fortran, and more recenly Rust (PyO3 is a great tool revolutionizing the field). To create something new, you need lower level programming, otherwise you stop at the *analyst* level.

> **i** Other scripting languages
>
> Python is mandatory, non-negotiable. But one should not limit themselves to a single scripting language. Knowning some shell automation, the basics of both Bash/other UNIX shell and PowerShell, are something you will need at some point. I also used to recommend Julia (its syntax is lovely), but recently I have some mixed feelings regarding its community, especially the way the company JuliaHub is handling its open source projects, which has become somewhat contradictory in terms of FOSS. Other languages - the *matlabish* (MATLAB, Octave, Scilab) environments - which are still used by *controls and automation* people, are mostly incompatible with good software practices and should be discouraged.

No need to tell you, at least a grasp of machine learning (ML) is required from a theoretical standpoint, as some practice with it in one of the above languages, specially Python (a minimal *scikit-learn* and PyTorch are highly desirable). Closely related to ML, in the field of statistics, you could also profit from knowning some R programming language. It is largely used by statisticians (especially in academic world), and the ecosystem around Tidyverse could be used in many other fields.

Back to the *documentation* world, typesetting languages such as Markdown, LaTeX, and Typst, can hugely increase productivity and compatibility with the aforementioned version controls. They support specific syntaxes for typing equations, and when you have a large number of equations, it becomes a nightmare (and it is also irrational) to use conventional text editors. Traditionally one would use LaTeX to create nicely looking reports and presentations (beamer), but the more recent Typst is sort of changing that by introducing modern practices into typesetting, and a blazing fast compilation time.

To wrap up, one needes to apply all the aforementioned skills to produce something useful. All of the above is worthless if unsupported by a domain-specific skill, *e.g.* CFD, DFT, MD, ML, ..., but that goes beyond our goal here to present those skills linked to the workflow, not the production itself.

Programming practices

At its core, serious *scientific computing* requires *scientific programming*. And good *scientific programming* is the least you need to demontrate respect for your pairs. Thus, it is not worth learning any programming before being introduced to the good practices. Many programmers write *garbage that works for them only*, and it is simply not possible to have a healthy collaboration with them. I often tell people not to ask me for help if they are not helping themselves, it is that simple - and bad coding is simply long term suicide.

One of the reasons Guido van Rossum created Python is because he wanted code to be readable. You should be able to guess what some code is doing even without specific technical knowledge about the language. This is probably the main feature that made its creation so popular in the scientific world.

Although they are applicable to Python, some practices introduced in the famous PEP8 can be extended to other languages; you should *read PEP8 religiously*. That document preconizes how to write clean and maintainable Python code. When transposing that to other languages, the minimum you are expected to do is:

- lines are limited to 79 characters
- use spaces around all operators
- consistent indentation with spaces

- blank lines around structural blocks
- lower case variable names
- Pascal-case structure names
- use underscore to separate words
- document functions properly

When you code, remember that most of the time what you are doing will be reviewed/used by somebody else and that person might not be in the mood to decipher the cryptic code you wrote; if that person is myself, *I will promptly refuse to help you* with badly written code. For newcomers, it is always better to talk about this before you write your first lines because once you stick to bad practices you will hardly ever leave them. Before you write something new, check if your mindset is also aligned with PEP20. Keep documenting your code as you write, what is generally done with Sphinx and some of the available syntax supports.

> ! For Julia users
>
> Julia has its own stylistic conventions that are simpler than PEP8; the main differences are the way to name functions (it recommends to *glue* words and use no underscore) and the *exclamation mark !* indicating a function modifies it(s) argument(s). For function naming you may chose to stick to PEP8 recommendation, what is my personal choice. The detailed document is found here.
>
> Julia also supports Unicode input, but its use is highly discouraged in modules. Unicode characters are better suited to write application scripts such as notebooks (in Pluto or Jupyter). For documentation, Julia has its own syntax which can be used to generate package documentation with help of Documenter.jl.

Python workflows

The evolution of Python package management has progressed from the foundational pip, through the era of Conda, to the modern dominance of uv. While `pip` remains the official tool for interacting with the Python Package Index (PyPI), its dependency resolution speed often poses limitations in complex projects. Conda gained significant popularity when Python expanded beyond POSIX-compliant environments to multi-platform systems, offering robust capabilities for environment isolation and package management, though its strong association with the Anaconda ecosystem eventually reduced its appeal. The landscape has been revolutionized by `uv`, a tool written in Rust that provides unparalleled speed in dependency resolution, environment creation, and environment recycling. Beyond pure package management, `uv` supports the creation of standard `pyproject.toml` configurations for non-package executable projects and integrates

natively with PyO3 to compile high-performance Rust extensions, establishing itself as the *de facto* standard for modern Python workflows as of 2026.

The scientific Python ecosystem relies on highly mature, core libraries that function as indispensable extensions of the language itself, most notably the **SciPy stack**, which encompasses **NumPy, Pandas, Matplotlib, SymPy**, and **SciPy**. For machine learning, workflows branch into classical approaches and deep learning. Classical machine learning and data preparation are dominated by **Scikit-learn**, which provides an extensive suite of standard algorithms, and **Scikit-image**, frequently employed for image analysis and dataset generation. When projects scale to require massive computational resources and neural networks, deep learning frameworks such as **PyTorch, TensorFlow**, and **JAX** are utilized. These tools are designed to maximize performance and provide seamless scalability directly within GPU memory.

Specialized scientific domains further rely on high-quality, academic-backed niche packages that serve as essential building blocks within the broader ecosystem. For instance, CasADi is heavily utilized in control, automation, and ordinary differential equations, enabling rapid prototyping of algorithmically differentiable systems and offering robust solvers for stiff equations. Similarly, Cantera serves as the foundational package for thermophysics, traditionally focused on gases but increasingly applied to other domains such as electrolytic cells and batteries. Because these niche tools are developed and maintained by dedicated academic research groups, they assure the long-term viability and continuous operation of these packages, allowing them to constitute critical building blocks of the basic Python ecosystem.

Producing reports

Upcoming

Operating systems

Upcoming